

MODULE 1

Detailed Day-by-Day Study Guide

Python Foundations · Linux Terminal · Git & GitHub

July 22 – August 1, 2026 | 10 Study Days | 3–4 Hours/Day

Projects: BMI Calculator (due Jul 28) · Password Generator (due Jul 31)

AWS SAA Exam Target: September 30, 2026

TIME	ACTIVITY
8:00 – 9:30 AM	New lesson + notes
9:30 – 9:45 AM	Break
9:45 – 11:15 AM	Coding lab / project work
11:15 – 11:45 AM	Linux terminal OR Git practice
Evening (opt.)	30 min GitHub commit + journal entry

DAY 1 · Wednesday, July 22

Python Setup + Variables + Data Types

Your first real day. Get everything installed and write your first script.

Environment Setup (Do This First — 30 min)

- Install Python 3.12+ from python.org (run: `python3 --version` to verify)
- Install VS Code from code.visualstudio.com
- Install VS Code extensions: Python (Microsoft), Pylance
- Open Terminal. Create your project folder: `mkdir cloud-engineering-labs && cd cloud-engineering-labs`
- Create your first Python file: `touch day1_variables.py` — open it in VS Code

Lesson: Variables and Data Types

A variable is a named container for a value. Python is dynamically typed — you don't declare the type, Python figures it out.

```
# The four core data types
name = 'Miryam' # str – text, always in quotes
age = 22 # int – whole number
height = 5.6 # float – decimal number
is_student = True # bool – True or False only

# print() displays output to the terminal
print(name) # Miryam
print(type(age)) #
print('My name is', name) # My name is Miryam

# String formatting – use f-strings (the modern way)
print(f'My name is {name} and I am {age} years old.')

# Basic math operators
print(10 + 3) # 13 addition
print(10 - 3) # 7 subtraction
print(10 * 3) # 30 multiplication
print(10 / 3) # 3.33 division (always float)
print(10 // 3) # 3 floor division (drops decimal)
print(10 % 3) # 1 modulo (remainder)
print(2 ** 8) # 256 exponent

# input() reads text from the user (always returns a string)
user_name = input('What is your name? ')
print(f'Hello, {user_name}!')

# Type conversion
num_str = '42'
num_int = int(num_str) # string -> int
numflt = float(num_str) # string -> float
back_str = str(num_int) # int -> string
```

■ Run each snippet in your terminal as you go. Don't just read — type it out.

Lab — Day 1 (Complete before tomorrow)

Create a file called `day1_lab.py` and write a program that:

- Asks the user for their name, age, and hometown using `input()`
- Stores each answer in a variable with a meaningful name
- Prints a formatted summary: 'Hi [name]! You are [age] years old and from [hometown].'
- Calculates what year the user was born ($2026 - \text{age}$) and prints it
- Uses `type()` to print the data type of each variable

■ Save the file. Run it with: `python3 day1_lab.py`

GitHub Setup (Do This Today)

- Create a GitHub account at github.com if you don't have one
- Create a new public repository called 'cloud-engineering-labs'
- Add a README.md with one line: 'Cloud Engineering Labs — by [Your Name]'
- Clone it locally: `git clone https://github.com/YOUR_USERNAME/cloud-engineering-labs.git`
- Copy your `day1_lab.py` file into the repo folder
- Run: `git add .` then `git commit -m 'feat: day 1 python variables lab'` then `git push`

■ **DEADLINE: First GitHub commit: TODAY, July 22**

DAY 2 · Thursday, July 23

Conditionals + Operators

Make your programs smart — teach them to make decisions.

Lesson: if / elif / else

Conditionals let your program take different paths based on data. Indentation is mandatory in Python — 4 spaces per level.

```
score = 85
if score >= 90:
    print('A')
elif score >= 80:
    print('B') # This runs — 85 >= 80
elif score >= 70:
    print('C')
else:
    print('Below C')

# Comparison operators
# == equal to != not equal to
# > greater than < less than
# >= greater or equal <= less or equal

# Logical operators – combine conditions
age = 20
has_id = True
if age >= 18 and has_id:
    print('Entry allowed')
if age < 13 or age > 65:
    print('Discount applies')
if not has_id:
    print('Please show ID')

# Truthy and Falsy – Python treats these as False:
# 0, '', [], {}, None, False
# Everything else is True
name = ''
if name:
    print('Name entered')
else:
    print('No name given') # This runs
```

Lab — Day 2

Create day2_lab.py. Write TWO programs:

Program 1 — Grade Calculator:

- Ask the user to enter a numeric score (0–100)
- Convert it to an integer with int()

- Use if/elif/else to print A, B, C, D, or F
- Also print: 'Pass' if score $\geq$ 60 else 'Fail'

Program 2 — FizzBuzz (classic interview problem):

- Print every number from 1 to 30
- If divisible by 3: print 'Fizz' instead
- If divisible by 5: print 'Buzz' instead
- If divisible by both 3 and 5: print 'FizzBuzz'
- Hint: use the modulo operator % to check divisibility

■ Commit to GitHub: `git commit -m 'feat: day 2 conditionals lab'`

DAY 3 · Friday, July 24

Loops — for, while, break, continue

Repetition is the engine of programming. Master loops today.

Lesson: for Loops

A for loop runs a block of code once for each item in a sequence.

```
# Loop over a range of numbers
for i in range(5): # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6): # 1, 2, 3, 4, 5 (start, stop)
    print(i)

for i in range(0, 20, 2): # 0, 2, 4 ... 18 (start, stop, step)
    print(i)

# Loop over a string
for letter in 'cloud':
    print(letter) # c l o u d (one per line)

# Loop over a list
languages = ['Python', 'Go', 'Rust']
for lang in languages:
    print(f'I want to learn {lang}')
```

Lesson: while Loops

A while loop runs as long as a condition is True. It is used when you don't know ahead of time how many times to loop.

```
count = 0
while count < 5:
    print(count)
    count += 1 # ALWAYS update the condition or you loop forever

# Common pattern: loop until valid input
while True:
    answer = input('Type yes or no: ')
    if answer in ['yes', 'no']:
        break # exits the loop
    print('Invalid input, try again')

# break – exit the loop immediately
# continue – skip to the next iteration
for i in range(10):
    if i == 5:
        break # stops at 5
    if i % 2 == 0:
        continue # skip even numbers
    print(i) # prints 1, 3
```

Lab — Day 3

Create day3_lab.py. Write TWO programs:

Program 1 — Multiplication Table:

- Ask the user for a number
- Use a for loop to print the multiplication table from 1 to 12
- Format it neatly: '7 x 1 = 7' ... '7 x 12 = 84'

Program 2 — Number Guessing Game:

- Import the random module: `import random`
- Generate a random number: `secret = random.randint(1, 100)`
- Use a while loop to keep asking the user to guess
- After each wrong guess, print 'Too high!' or 'Too low!'
- When they guess correctly, print the number of tries it took

■ *This is the most fun lab so far — enjoy it. Commit when done.*

DAY 4 · Saturday, July 25

Functions — def, parameters, return

Functions are how you organize code so it can be reused. Start your BMI Calculator.

Lesson: Functions

A function is a named, reusable block of code. You define it once and call it as many times as you need.

```
# Basic function – no parameters, no return value
def greet():
    print('Hello, Cloud Engineer!')
greet() # call the function

# Function with parameters
def greet_user(name):
    print(f'Hello, {name}!')
greet_user('Miryam')

# Function with return value
def add(a, b):
    return a + b
result = add(10, 5)
print(result) # 15

# Default parameter values
def power(base, exponent=2): # exponent defaults to 2
    return base ** exponent
print(power(3)) # 9 (uses default)
print(power(3, 3)) # 27 (overrides default)

# Functions can call other functions
def square(n):
    return n * n
def sum_of_squares(a, b):
    return square(a) + square(b)
print(sum_of_squares(3, 4)) # 25
```

■ *Rule: a function should do ONE thing and do it well. If it does two things, split it into two functions.*

Lab — Day 4 + START Project 1: BMI Calculator

Create `day4_functions.py` and practice writing 3 small functions:

- A function `celsius_to_fahrenheit(c)` that converts temperature
- A function `is_even(n)` that returns `True` if `n` is even, `False` if odd
- A function `count_vowels(word)` that counts vowels in a string

Then create `bmi_calculator.py` — your first real project:

- Write a function `calculate_bmi(weight_kg, height_m)` that returns `weight / height**2`
- Write a function `get_bmi_category(bmi)` that returns `Underweight / Normal / Overweight / Obese`
- Ask the user for weight (kg) and height (m) using `input()`
- Call your functions, print the BMI rounded to 1 decimal, and print the category

- Add an option: ask if they want to enter weight in lbs and height in inches, then convert

■ **DEADLINE: BMI Calculator — COMPLETE by Tuesday July 28**

■ *You've now done 4 days. Sunday is rest. Come back Monday refreshed.*

SUNDAY · July 26 — REST DAY

Light review only · Organize your GitHub · Plan the week ahead

Rest Day tasks (light, 30 min max):

- Review your GitHub — make sure days 1–4 are committed and pushed
- Read your notes from the week — highlight anything confusing
- Plan Monday: re-read the Day 5 section of this guide
- Optional: watch a 10-min YouTube video on Python lists to preview tomorrow

DAY 5 · Monday, July 27

Lists — The Most Used Data Structure in Python

Lists are everywhere. Master them completely before moving on.

Lesson: Creating and Accessing Lists

```
# A list holds multiple items in order, in square brackets
skills = ['Python', 'AWS', 'Linux', 'Git', 'Docker']
numbers = [1, 2, 3, 4, 5]
mixed = [42, 'hello', True, 3.14] # lists can hold any types
empty = [] # empty list

# Indexing – starts at 0
print(skills[0]) # 'Python' (first item)
print(skills[-1]) # 'Docker' (last item)
print(skills[-2]) # 'Git' (second from last)

# Slicing – [start:stop:step] (stop is NOT included)
print(skills[1:3]) # ['AWS', 'Linux']
print(skills[:2]) # ['Python', 'AWS'] (from beginning)
print(skills[3:]) # ['Git', 'Docker'] (to end)
print(skills[::-1]) # reverse the list
```

Lesson: List Methods (The Important Ones)

```
languages = ['Python', 'Go', 'Java']
languages.append('Rust') # add to end: ['Python', 'Go', 'Java', 'Rust']
languages.insert(1, 'C++') # insert at index 1
languages.remove('Go') # remove by value
popped = languages.pop() # remove and return last item
popped = languages.pop(0) # remove and return item at index 0

nums = [3, 1, 4, 1, 5, 9, 2, 6]
nums.sort() # sorts in place: [1,1,2,3,4,5,6,9]
nums.sort(reverse=True) # descending
nums.reverse() # reverses in place

print(len(languages)) # number of items
print('Python' in languages) # True – membership test
print(languages.count('Python')) # count occurrences
languages.clear() # empties the list

# Iterating
for skill in skills:
    print(skill)

# enumerate gives you index AND value
for i, skill in enumerate(skills):
    print(f'{i+1}. {skill}')
```

Lab — Day 5

Create day5_lists.py. Build a simple To-Do List program:

- Start with an empty list called tasks = []
- Show a menu: 1=Add task, 2=View tasks, 3=Remove task, 4=Quit
- Use a while loop to keep showing the menu
- Add tasks with append(), view with a for loop + enumerate, remove with remove()
- Handle the case where the user tries to remove a task that doesn't exist (try/except or 'in' check)

■ Commit: 'feat: day 5 lists lab — todo app'

DAY 6 · Tuesday, July 28

Dictionaries + Complete BMI Calculator

Dictionaries label your data. Project 1 is due today.

Lesson: Dictionaries

A dictionary stores key-value pairs. Keys must be unique. Access by key, not by index.

```
# Creating a dictionary
engineer = {
    'name': 'Miryam',
    'role': 'Cloud Engineer',
    'skills': ['Python', 'AWS', 'Terraform'],
    'certified': False
}

# Access values by key
print(engineer['name']) # Miryam
print(engineer.get('role')) # Cloud Engineer
print(engineer.get('salary', 0)) # 0 (default if key missing)

# Add or update
engineer['certified'] = True
engineer['salary'] = 90000

# Delete
del engineer['salary']

# Looping
for key in engineer:
    print(key, ': ', engineer[key])
for key, value in engineer.items():
    print(f'{key} -> {value}')

# Useful methods
print(engineer.keys()) # all keys
print(engineer.values()) # all values
print('name' in engineer) # True – key exists check
```

Project 1 — BMI Calculator (DUE TODAY)

■ DEADLINE: BMI Calculator: DEADLINE July 28

Your bmi_calculator.py must include ALL of the following:

- Function calculate_bmi(weight_kg, height_m) — returns float
- Function get_bmi_category(bmi) — returns string category
- Imperial unit support (lbs/inches with conversion to metric)
- Input validation: reject negative numbers or zero height
- A dictionary storing BMI thresholds: {'Underweight': 18.5, 'Normal': 24.9, ...}
- Clear, formatted output with the BMI value and category
- A professional README.md in the project folder

After completing, push to GitHub:

```
git add bmi_calculator.py README.md
git commit -m 'feat: project 1 BMI calculator complete'
git push
```

Lab — Day 6 (After Project)

Create day6_dicts.py. Build a simple phone book:

- contacts = {} (empty dictionary)
- Menu: 1=Add contact, 2=Search, 3=Delete, 4=List all, 5=Quit
- Store name as key, phone number as value
- Handle 'contact not found' gracefully

■ Commit: 'feat: day 6 dictionaries lab — phonebook'

DAY 7 · Wednesday, July 29

Linux Terminal Basics

The terminal is your home as a cloud engineer. Get comfortable in it.

Why Linux Matters for Cloud Engineers

Every AWS EC2 instance, every Docker container, every Kubernetes pod runs Linux. If you cannot navigate the terminal, you cannot do the job. Practice these commands until they are muscle memory.

Essential Commands — Run Every Single One Today

COMMAND	WHAT IT DOES	EXAMPLE
pwd	Print current directory	pwd
ls	List files	ls -la
cd	Change directory	cd ~/Documents
mkdir	Make a directory	mkdir my_project
touch	Create an empty file	touch notes.txt
cat	Display file contents	cat notes.txt
echo	Print text or write to file	echo 'hello' > file.txt
cp	Copy file or folder	cp file.txt backup.txt
mv	Move or rename	mv old.txt new.txt
rm	Delete file	rm -rf folder/
nano	Simple text editor	nano notes.txt
man	Show manual for a command	man ls
grep	Search inside files	grep 'error' log.txt
history	Show command history	history grep python
clear	Clear the terminal screen	clear
chmod	Change file permissions	chmod 755 script.sh
ls -la	List with permissions + hidden	ls -la ~/
~	Home directory shortcut	cd ~
..	Go up one directory	cd ../..

Lab — Day 7

Open your terminal and complete all of the following — no skipping:

- Navigate to your home directory. Create a folder structure: projects/cloud-labs/python/
- Inside python/, create 3 files: hello.py, notes.txt, and scratch.py using touch
- Use echo to write 'My first Python file' into notes.txt. Verify with cat.
- Copy notes.txt to a backup: notes_backup.txt
- Use ls -la to see permissions on all files in the folder
- Use chmod 755 hello.py and verify it changed with ls -la
- Use grep to search for 'Python' in notes.txt

- Delete scratch.py with rm. Verify it's gone with ls.
- Practice navigating with cd ../../.. and back again using cd -

■ *No GitHub commit needed today — just terminal practice. But do make a note of what you learned.*

DAY 8 · Thursday, July 30

Git & GitHub — Professional Version Control

Git is non-negotiable. Every cloud job expects it on day one.

Lesson: Core Git Concepts

Git tracks changes to your code over time. Every saved snapshot is a commit. GitHub hosts your repos online.

```
# ■■ First-time setup (run once) ■■
git config --global user.name 'Your Name'
git config --global user.email 'your@email.com'
git config --global core.editor 'code --wait' # use VS Code as editor

# ■■ Starting a project ■■
git init # initialize a new repo in current folder
git clone # copy an existing repo from GitHub

# ■■ The daily workflow ■■
git status # see what changed
git add . # stage ALL changes
git add filename.py # stage one specific file
git commit -m 'feat: add BMI calculator' # save snapshot
git push # upload to GitHub
git pull # download latest from GitHub

# ■■ Viewing history ■■
git log # full history
git log --oneline # compact history
git diff # see unstaged changes

# ■■ Branches ■■
git branch # list branches
git checkout -b feature/bmi-calculator # create and switch to new branch
git checkout main # switch back to main
git merge feature/bmi-calculator # merge branch into main

# ■■ Undo mistakes ■■
git restore filename.py # discard unstaged changes to a file
git reset HEAD~1 # undo last commit (keeps changes)
```

The .gitignore File

This file tells Git to ignore certain files — secrets, caches, virtual environments.

```
# Create a .gitignore in your project root:
__pycache__/
*.pyc
.env # NEVER commit secrets
venv/
.DS_Store # macOS file
*.tfstate # Terraform state files (later)
```

Conventional Commits — Write These From Day 1

PREFIX	USE WHEN
feat:	Adding new functionality
fix:	Fixing a bug
docs:	Updating README or documentation
chore:	Maintenance (updating .gitignore, config)
test:	Adding tests
refactor:	Improving code without changing behavior

Lab — Day 8

- Create a new branch: `git checkout -b feature/password-generator`
- Start your `password_generator.py` file (Project 2 — due tomorrow)
- Commit the initial file: `git commit -m 'feat: start password generator project'`
- Merge back to main, push, and verify on GitHub.com that you see the commit
- Add a `.gitignore` to cloud-engineering-labs with the entries above

DAY 9 · Friday, July 31

Password Generator + Portfolio Push

Project 2 is due today. Push everything. Module 1 wraps up.

Project 2 — Password Generator (DUE TODAY)

■ DEADLINE: Password Generator: DEADLINE July 31

Create password_generator.py. Requirements:

- Import the random and string modules (both built into Python)
- Function generate_password(length, use_upper, use_digits, use_symbols) that returns a password string
- string.ascii_lowercase, string.ascii_uppercase, string.digits, string.punctuation — use these
- Ask the user for password length and which character types to include
- Validate: minimum length of 8, reject if no character types selected
- Ask how many passwords to generate and print them all
- Write a README.md explaining what the program does and how to run it

Starter structure:

```
import random
import string

def generate_password(length, use_upper=True, use_digits=True, use_symbols=True):
    chars = string.ascii_lowercase
    if use_upper: chars += string.ascii_uppercase
    if use_digits: chars += string.digits
    if use_symbols: chars += string.punctuation
    return ''.join(random.choice(chars) for _ in range(length))

# main code
length = int(input('Password length: '))
password = generate_password(length)
print(f'Generated: {password}')
```

Final Push — Everything to GitHub

Before end of day today, your GitHub must show:

- cloud-engineering-labs/day1_lab.py
- cloud-engineering-labs/day2_lab.py
- cloud-engineering-labs/day3_lab.py
- cloud-engineering-labs/day4_functions.py
- cloud-engineering-labs/day5_lists.py
- cloud-engineering-labs/day6_dicts.py
- cloud-engineering-labs/bmi_calculator.py + README.md
- cloud-engineering-labs/password_generator.py + README.md
- cloud-engineering-labs/.gitignore

```
git add .
git commit -m 'feat: project 2 password generator complete — module 1 done'
git push
```

■ Go to your GitHub profile in a browser. Verify the green contribution squares are showing. Screenshot it — you earned it.

DAY 10 · Saturday, August 1

Week Review + Mock Interview

Consolidate everything. Practice explaining your work out loud.

Review Checklist — Work Through Each One

TOPIC	CAN YOU DO IT? (Write Yes / Not Yet / Need Review)
Declare variables of all 4 core types	
Write and call a function with parameters and a return value	
Use a for loop with range() and with a list	
Write a while loop with a break condition	
Create, access, slice, and modify a list	
Create a dictionary and loop over .items()	
Write a complete if/elif/else block	
Navigate the Linux file system using the terminal only	
Create a .gitignore and explain why secrets go in it	
Run the full git workflow: add → commit → push	
Explain what a branch is and why you'd use one	
Explain your BMI Calculator project in 2 minutes	
Explain your Password Generator project in 2 minutes	

HackerRank Practice — Do These Today

Go to hackerrank.com/domains/python — complete these easy problems:

- Say Hello, World! With Python
- Python If-Else
- Arithmetic Operators
- Python: Division
- Loops
- Write a Function (is_leap_year)

Module 1 Mock Interview — Answer Out Loud, Not Just in Your Head

Q: Tell me about yourself and your technical background.

A: Keep it under 90 seconds. Structure: current situation → what you've built in the last 10 days (BMI Calculator, Password Generator, pushed to GitHub) → what you're working toward (AWS cert, cloud engineering role). End with why you're excited about this company/role.

Q: What is the difference between a list and a dictionary in Python?

A: A list is an ordered collection accessed by numeric index — `list[0]` gives the first item. A dictionary is an unordered collection of key-value pairs accessed by key — `dict['name']`. Use a list when order matters and items don't need labels. Use a dictionary when you want to label and look up data by a meaningful name.

Q: Explain what Git is and walk me through your typical workflow.

A: Git is a version control system that tracks changes to code over time. My daily workflow: I create a feature branch, make changes, use `'git add'` to stage them, `'git commit -m'` to save a snapshot with a descriptive message, then `'git push'` to upload to GitHub. If working with others I open a pull request for review before merging to main.

Q: What does the 'return' statement do in a Python function?

A: It sends a value back to wherever the function was called from, and also exits the function immediately. Without return, a function returns None by default. For example, my `calculate_bmi()` function returns the computed float value so I can store it in a variable and use it elsewhere.

Q: Walk me through one of your projects on GitHub.

A: 'I built a BMI Calculator in Python. The user enters their weight and height — in either metric or imperial units — and the program calculates their BMI using the formula $\text{weight} / \text{height squared}$, then tells them their category: Underweight, Normal, Overweight, or Obese. I used functions to keep the logic separated, a dictionary to store the BMI thresholds, and input validation to reject invalid entries. The code is on my GitHub with a README.' Be specific and confident.

Q: What Linux command would you use to see who owns a file and its permissions?

A: `ls -la` shows a detailed listing including permissions (like `-rwxr-xr-x`), the owner, group, file size, and modification date. The permission string breaks down as: first character is file type, next three are owner permissions, next three are group, last three are others.

Q: What's the difference between 'git add' and 'git commit'?

A: `'git add'` stages your changes — it marks specific files to be included in the next commit. Think of it like placing items in a box. `'git commit'` then seals that box and saves it permanently to the local history with a message. You can stage multiple times before committing to group related changes together.

What's Coming in Module 2 — Preview

Starting Monday August 3, you enter Module 2: Python Intermediate + AWS Cloud Practitioner.

- Python: file I/O, exception handling, JSON, APIs, classes
- AWS: IAM, EC2, S3, VPC, RDS, Lambda, CloudWatch (Cloud Practitioner exam coverage)
- Projects: Expense Tracker, Contact Book, Weather API App, S3 Static Website, EC2 Web Server
- BIG deadline: AWS Cloud Practitioner exam — August 31, 2026
- AWS SAA exam target: September 30, 2026

Module 1 complete. You have a GitHub portfolio, two real projects, and the fundamentals of Python, Linux, and Git. That is more than most people will ever start.